

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN - ĐIỆN TỬ



MÔN HỌC

TRÍ TUỆ NHÂN TẠO TRONG ĐIỀU KHIỂN

"THUẬT TOÁN PLA - PERCEPTION LEARNING ALGORITHM"

REPORT HOMEWORK 1

Lớp: L01 Nhóm: 05 HK261

GVHD: TS. PHẠM VIỆT CƯỜNG

STT	Sinh viên	MSSV	Điểm
1	Nguyễn Văn Tuấn Anh	2310131	
2			
3			
4			

TP. Hồ Chí Minh, ngày 23 tháng 09 năm 2026

Mục lục

1	Lý thuyết	2
1.1	Phân lớp nhị phân	2
1.2	Thuật toán perceptron	2
1.2.1	Cách phân lớp	2
1.2.2	Thuật toán phân lớp Perceptron	3
2	Giải thích mã nguồn	3
2.1	Khối 1: Sinh hai lớp điểm	3
2.2	Khối 2: Các hàm của PLA	4
2.2.1	Hàm lấy dấu	4
2.2.2	Dự đoán một điểm	4
2.2.3	Dự đoán cả tập	5
2.2.4	Kiểm tra hội tụ	5
2.2.5	Vòng lặp cập nhật	5
2.3	Khối 3: Chạy PLA	6
2.4	Khối 4: Vẽ đường phân tách	7
2.5	Khối 5: Hoạt họa từng lần cập nhật	7

Danh sách hình vẽ

1	Bài toán phân lớp nhị phân trong 2D	2
---	---	---

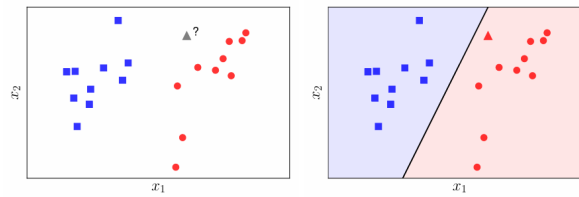
1 Tóm tắt lý thuyết

Chương này trình bày bài toán phân lớp nhị phân và thuật toán học Perceptron (Perceptron Learning Algorithm, PLA).

1.1 Phân lớp nhị phân

Bài toán phân lớp nhị phân là phân loại 2 lớp dữ liệu riêng biệt (hay còn gọi là binary classification).

Giả sử có hai class tương ứng hai tập điểm xanh và đỏ, bài toán đặt ra là từ dữ liệu đã gán nhãn xanh-đỏ (cho rằng tập 2 dữ liệu này linearly separable), ta cần tìm một bộ phân lớp (classifier) có khả năng dự đoán màu của một điểm dữ liệu mới.



Hình 1: Bài toán phân lớp nhị phân trong 2D

Nếu coi mỗi vector đặc trưng là một điểm trong không gian d chiều, bài toán phân lớp coi như bài toán xác định điểm đó thuộc lớp nào. Hay nói cách khác, nếu mỗi lớp chiếm một vùng trong không gian, ta cần xác định ranh giới (boundary) giữa các vùng này.

1.2 Thuật toán perceptron

1.2.1 Cách phân lớp

- Giả sử $X = [x_1, x_2, \dots, x_n] \in \mathbb{R}^{d \times N}$ là ma trận vector chứa các điểm huấn luyện.
 - x_i là vector đặc trưng thứ i
 - Nhãn $y = [y_1, y_2, \dots, y_n] \in \mathbb{R}^{1 \times N}$ với $y_i = 1$ nếu x_i thuộc lớp 1, $y_i = -1$ nếu x_i thuộc lớp 2
- Tại một thời điểm, giả sử có boundary là siêu phẳng có pt:

$$f(\mathbf{x}) = w_1x_1 + \dots + w_dx_d + w_0 = \mathbf{w}^T \mathbf{x} + w_0 = 0$$

- Với $\mathbf{w} \in \mathbb{R}^d$ là vector hệ số, w_0 là bias (tự do).
- Dùng **bias trick** để đưa w_0 vào $\mathbf{w}$, ta có siêu phẳng là

$$\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad \tilde{\mathbf{w}} = \begin{bmatrix} \mathbf{w} \\ w_0 \end{bmatrix}, \quad f(\tilde{\mathbf{x}}) = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}} = 0$$

- Nếu $\mathbf{w}$ là nghiệm của bài toán perceptron, với data mới $\mathbf{x}$ chưa gán nhãn, ta có thể gán nhãn bằng cách:

$$\text{label}(\mathbf{x}) = \begin{cases} 1 & \text{nếu } \mathbf{w}^T \mathbf{x} \geq 0, \\ -1 & \text{o.w.} \end{cases}$$

- Nói cách khác, $\text{label}(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x})$ với sgn là hàm xác định dấu, giả sử rằng $\text{sgn}(0) = 1$.

1.2.2 Thuật toán phân lớp Perceptron

Bằng việc xây dựng hàm mất mát:

$$J(\mathbf{w}) = \sum_{\mathbf{x}_i \in \mathcal{M}} (-y_i \mathbf{w}^T \mathbf{x}_i)$$

Trong đó $\mathcal{M}$ là tập các điểm dữ liệu bị phân lớp lỗi, tức là các điểm dữ liệu mà $\text{label}(\mathbf{x}_i) \neq y_i$.

Và với một điểm dữ liệu $\mathbf{x}_i$ bị phân lớp lỗi, hàm mất mát và đạo hàm của nó:

$$J(\mathbf{w}; \mathbf{x}_i, y_i) = -y_i \mathbf{w}^T \mathbf{x}_i, \quad \nabla_{\mathbf{w}} J(\mathbf{w}; \mathbf{x}_i, y_i) = -y_i \mathbf{x}_i$$

Và tối ưu hàm mất mát bằng Schoolastic Gradient Descent (SGD):

$$\mathbf{w} \leftarrow \mathbf{w} - \eta (-y_i \mathbf{x}_i) = \mathbf{w} + \eta y_i \mathbf{x}_i$$

2 Giải thích mã nguồn

Chương này đi theo src/basic.py. Tập gồm năm khối, đúng với các mục comment trong mã: sinh dữ liệu, các hàm của PLA, chạy thuật toán, vẽ đường phân tách, và hoạt họa từng lần cập nhật. Mỗi mục dưới đây trích đúng khối tương ứng rồi giải thích khối đó làm gì.

2.1 Khối 1: Sinh hai lớp điểm

Ba dòng đầu nhập NumPy để tính trên mảng, Matplotlib để vẽ, và FuncAnimation để xuất từng khung hình. Phần sinh dữ liệu là Listing 1. Số dòng in bên cạnh mã trùng với số dòng trong src/basic.py.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from matplotlib.animation import FuncAnimation
4
5 # =====
6 # 1. create 2 classes of 2D points
7 # =====
8 np.random.seed(2)
9
10 # center values for 2 classes
11 mean0 = [2, 2] # +1
12 mean1 = [4, 2] # -1
13
14 # co-variance
15 cov = [[0.3, 0.2],
16        [0.2, 0.3]]
17
18 N = 5 # number of points per class
19
20 # create random points for each class following Gaussian distribution
21 # N rows, 2 columns (each row is a point x, y)
22 class0 = np.random.multivariate_normal(mean0, cov, N)
23 class1 = np.random.multivariate_normal(mean1, cov, N)
24
25 # convert to shape (2 rows, N columns) for easier processing
26 X0 = class0.T
27 X1 = class1.T
28
29 # combine the two classes into one dataset
30 # X_points shape = (2, 40)
31 X_points = np.concatenate((X0, X1), axis=1)
32
```

```
33 # labels for the two classes: +1 for class0, -1 for class1
34 # y shape = (1, 40)
35 y_pos = np.ones((1, N))
36 y_neg = -1 * np.ones((1, N))
37 y = np.concatenate((y_pos, y_neg), axis=1)
38
39 # add a row of ones to X_points to create X_bar for bias term
40 # X shape = (3, 40): row 0 = 1, row 1 = x, row 2 = y
41 ones_row = np.ones((1, 2 * N))
42 X = np.concatenate((ones_row, X_points), axis=0)
```

Listing 1: Sinh hai lớp điểm trên mặt phẳng và ghép thành ma trận học.

`np.random.seed(2)` cố định chuỗi số ngẫu nhiên. Mọi lần chạy sau đó tạo cùng tập điểm, cùng trọng số ban đầu và cùng thứ tự xáo trộn trong PLA.

Hai lớp là hai đám điểm Gauss hai chiều. Lớp $+1$ có tâm $[2, 2]$, lớp -1 có tâm $[4, 2]$. Cùng ma trận hiệp phương sai

$$\begin{bmatrix} 0.3 & 0.2 \\ 0.2 & 0.3 \end{bmatrix}$$

nên mỗi đám hơi kéo theo đường chéo, nhưng hai tâm vẫn cách nhau theo phương ngang. Mỗi lớp lấy $N = 5$ điểm, nên cả tập có 10 điểm.

`multivariate_normal` trả về mảng cỡ $(N, 2)$: mỗi hàng là một điểm. Mã chuyển vị thành X_0, X_1 cỡ $(2, N)$ để mỗi cột là một điểm. Cách xếp này giúp lấy điểm thứ i bằng $X[:, i]$.

`X_points` ghép hai lớp theo cột, cỡ $(2, 10)$. Nhân được ghép tương ứng: năm số $+1$ rồi năm số -1 , nên `y` cỡ $(1, 10)$. Comment trong tệp ghi shape $(2, 40)$ và $(1, 40)$; đó là dấu vết của một ví dụ cũ với nhiều điểm hơn. Với $N = 5$ đang dùng, số cột đúng là $2N = 10$.

Hàng toàn số 1 được ghép lên trên `X_points`, tạo ra `X` cỡ $(3, 10)$. Hàng 0 là hằng số 1 của hệ số tự do, hàng 1 là hoành độ, hàng 2 là tung độ. Đây chính là các vector $\bar{x}$ ở (??), xếp thành các cột.

2.2 Khối 2: Các hàm của PLA

Khối này gồm năm hàm. Chúng tách riêng phép lấy dấu, dự đoán một điểm, dự đoán cả tập, kiểm tra hội tụ và vòng lặp cập nhật.

2.2.1 Hàm lấy dấu

```
48 def sign_of(value):
49     """Giống np.sign: > 0 -> 1, < 0 -> -1, = 0 -> 0."""
50     if value > 0:
51         return 1
52     if value < 0:
53         return -1
54     return 0
```

Listing 2: Hàm dấu dùng trong luật dự đoán.

`sign_of` làm đúng quy ước `sign` ở chương ??: dương ra 1, âm ra -1 , bằng 0 thì ra 0. Không gộp trường hợp 0 vào một trong hai lớp, vì điểm nằm trên biên chưa được phân loại đúng.

2.2.2 Dự đoán một điểm

```
57 def predict_one(w, x):
58     """
59     Dự đoán nhãn của 1 điểm.
60     w: vector trọng số, shape (3, 1)
61     x: 1 điểm [1, x, y], shape (3, 1)
```

```
62
63     Tinh tich vo huong: w0*1 + w1*x + w2*y
64     roi lay dau.
65     """
66     s = 0.0
67     for k in range(len(x)):
68         s = s + w[k, 0] * x[k, 0]
69     return sign_of(s)
```

Listing 3: Tính $\text{sign}(\mathbf{w}^\top \mathbf{x})$ cho một cột của $\mathbf{X}$.

$\mathbf{w}$ và $\mathbf{x}$ đều có shape (3, 1). Vòng lặp cộng $w_0 \cdot 1 + w_1 x_1 + w_2 x_2$, tức tích vô hướng $\mathbf{w}^\top \mathbf{x}$, rồi trả về dấu của tổng. Hàm không dùng np.dot để từng số hạng của công thức (??) hiện rõ trong mã.

2.2.3 Dự đoán cả tập

```
72 def predict_all(w, X_data):
73     """Du doan nhan tat ca cac diem."""
74     n_points = X_data.shape[1]
75     preds = []
76     for i in range(n_points):
77         xi = X_data[:, i].reshape(3, 1)
78         preds.append(predict_one(w, xi))
79     return preds
```

Listing 4: Áp dụng dự đoán cho mọi cột của ma trận dữ liệu.

$\mathbf{X_data.shape[1]}$ là số điểm. Cột i được reshape(3, 1) rồi đưa vào predict_one. Kết quả là một danh sách Python dài bằng số điểm, không phải mảng NumPy; các hàm phía sau chỉ cần so sánh từng phần tử với nhãn.

2.2.4 Kiểm tra hội tụ

```
82 def has_converged(X_data, y_data, w):
83     """True neu moi diem deu duoc phan loai dung."""
84     preds = predict_all(w, X_data)
85     n_points = X_data.shape[1]
86     for i in range(n_points):
87         if preds[i] != y_data[0, i]:
88             return False
89     return True
```

Listing 5: Dừng khi không còn điểm nào bị gán sai nhãn.

Hàm trả về True chỉ khi mọi dự đoán trùng nhãn. So sánh là $\neq$, nên dự đoán 0 không khớp +1 hay -1 và bị tính là chưa hội tụ. Đây đúng định nghĩa tách được tuyến tính ở chương trước: tồn tại $\mathbf{w}$ sao cho dấu của $\mathbf{w}^\top \mathbf{x}_i$ bằng y_i với mọi i .

2.2.5 Vòng lặp cập nhật

```
92 def perceptron(X_data, y_data, w_init):
93     """
94     Perceptron Learning Algorithm (PLA).
95
96     Moi vong:
97     - xao tron thu tu cac diem
98     - neu diem bi sai: w_new = w_old + y * x
99     Lap den khi khong con diem sai.
100     """
101     w_history = [w_init]
102     mis_points = []
103     n_points = X_data.shape[1]
```

```
104
105 while True:
106     # xáo trộn thứ tự điểm
107     order = np.random.permutation(n_points)
108
109     for i in range(n_points):
110         idx = order[i]
111         xi = X_data[:, idx].reshape(3, 1)
112         yi = y_data[0, idx]
113
114         pred = predict_one(w_history[-1], xi)
115         if pred != yi:
116             # điểm này đang bị phân loại sai -> cập nhật w
117             mis_points.append(idx)
118             w_new = w_history[-1] + yi * xi
119             w_history.append(w_new)
120
121         if has_converged(X_data, y_data, w_history[-1]):
122             break
123
124     return w_history, mis_points
```

Listing 6: PLA xáo trộn thứ tự điểm và sửa trọng số mỗi lần dự đoán sai.

$w_history$ bắt đầu bằng trọng số khởi tạo. mis_points ghi chỉ số của những điểm gây ra một lần cập nhật, theo đúng thứ tự các lần đó xảy ra.

Mỗi vòng `while` xáo một hoán vị của $0, 1, \dots, 9$. Với từng chỉ số trong hoán vị, chương trình lấy cột tương ứng của X và nhân ở hàng 0 của y . Nếu dấu dự đoán khác nhân, điểm đó được ghi vào mis_points và trọng số được sửa bằng

$$w_{\text{mới}} = w_{\text{cũ}} + y_i \bar{x}_i,$$

đúng luật (??). Vector mới được nối vào cuối $w_history$.

Kiểm tra hội tụ nằm *sau* cả lượt duyệt, không nằm ngay sau từng điểm. Vì vậy một lượt có thể sửa trọng số nhiều lần. Khi `has_converged` đúng, hàm trả về toàn bộ lịch sử trọng số và danh sách điểm đã gây cập nhật. Nếu dữ liệu không tách được tuyến tính, `while True` không thoát.

2.3 Khối 3: Chạy PLA

```
128 # 3. Chạy PLA
129 # =====
130 d = X.shape[0] # = 3
131 w_init = np.random.randn(d, 1)
132 w_history, mis_points = perceptron(X, y, w_init)
133 print(mis_points)
```

Listing 7: Khởi tạo trọng số ngẫu nhiên và chạy PLA trên tập vừa sinh.

$d = 3$ là số hàng của X . Trọng số ban đầu là một cột Gauss chuẩn, shape $(3, 1)$. Vì seed đã đặt ở khối 1, vector này không đổi giữa các lần chạy.

Với seed 2, chương trình in ra 20 chỉ số điểm bị phân loại sai trước khi hội tụ:

$[3, 8, 0, 9, 4, 3, 6, 2, 7, 1, 6, 4, 8, 0, 5, 1, 5, 1, 8, 4]$.

Chỉ số 0 đến 4 thuộc lớp $+1$, chỉ số 5 đến 9 thuộc lớp -1 . Cả hai lớp đều xuất hiện trong danh sách, nghĩa là biên ban đầu cắt qua cả hai đám điểm và phải được kéo dần. $w_history$ có 21 phần tử: một vector khởi tạo cộng 20 lần cập nhật.

2.4 Khối 4: Vẽ đường phân tách

```

139 def draw_line(w):
140     """Vẽ đoạn thẳng  $w_0 + w_1x + w_2y = 0$  trong cửa sổ đồ thị."""
141     w0, w1, w2 = [float(v) for v in w.ravel()]
142
143     if w2 != 0:
144         x_left = 0.0
145         x_right = 6.0
146         y_left = -(w1 * x_left + w0) / w2
147         y_right = -(w1 * x_right + w0) / w2
148         return plt.plot([x_left, x_right], [y_left, y_right], "k-")
149
150     # dừng nếu  $w_2 = 0$ 
151     x_vert = -w0 / w1
152     return plt.plot([x_vert, x_vert], [-2.0, 4.0], "k-")

```

Listing 8: Đổi phương trình biên thành đoạn thẳng trong cửa sổ đồ thị.

Đường cần vẽ vẫn là $w_0 + w_1x + w_2y = 0$. Khi $w_2 \neq 0$, giải tung độ

$$y = -\frac{w_1x + w_0}{w_2}$$

tại hai mép $x = 0$ và $x = 6$, đúng bề rộng cửa sổ sẽ dùng ở khối 5. Không còn vẽ đoạn từ -100 đến 100 rồi để trục cắt bớt.

Nếu $w_2 = 0$, biên là đường đứng $x = -w_0/w_1$, vẽ trong khoảng y từ -2 đến 4 .

2.5 Khối 5: Hoạt họa từng lần cập nhật

```

158 def viz_alg_1d_2(w_list):
159     n_iter = len(w_list)
160     plt.figure()
161
162     def update(i):
163         plt.cla()
164         plt.plot(X0[0, :], X0[1, :], "b^", label="+1")
165         plt.plot(X1[0, :], X1[1, :], "ro", label="-1")
166         plt.xlim(0, 6)
167         plt.ylim(-2, 4)
168         plt.xlabel("x")
169         plt.ylabel("y")
170         plt.grid(True)
171         plt.legend()
172
173         draw_line(w_list[i])
174
175     # khoanh điểm đang bị sửa, tru frame cuối
176     if i < n_iter - 1:
177         px = float(X[1, mis_points[i]])
178         py = float(X[2, mis_points[i]])
179         plt.plot(px, py, "ko", markersize=12, fillstyle="none")
180
181     plt.title("PLA: bước %d/%d" % (i, n_iter - 1))
182
183     anim = FuncAnimation(
184         plt.gcf(), update, frames=n_iter, interval=800, repeat=False
185     )
186     anim.save("pla_vis.gif", writer="pillow")
187     plt.show()
188

```

```
189 |  
190 | viz_alg_1d_2(w_history)
```

Listing 9: Mỗi khung hình là một trọng số, vẽ bằng giao diện Matplotlib mặc định.

viz_alg_1d_2 nhận w_history. n_iter là số vector trọng số, bằng 21 với seed 2.

Mỗi khung chỉ dùng các lệnh vẽ cơ bản:

- Tam giác xanh là lớp +1, chấm đỏ là lớp -1. Chú giải nằm ở góc hình.
- Trục hiện số, có nhãn x, y , lưới và giới hạn $[0, 6] \times [-2, 4]$. Không ẩn vạch chia.
- Khung i vẽ đúng w thứ i . Với $i < 20$, một vòng tròn rỗng khoanh điểm vừa bị phân loại sai. Tọa độ lấy ở hàng 1 và hàng 2 của X. Khung cuối chỉ còn đường thẳng đã hội tụ.
- Tiêu đề ghi bước hiện tại trên tổng số 20 lần cập nhật.

FuncAnimation chạy đúng 21 khung, mỗi khung 800ms, rồi ghi pla_vis.gif. Không còn hai khung thừa để giữ hình cuối.

Nhìn lại cả năm khối, dữ liệu ở khối 1 tạo đúng cặp $(\bar{x}, y)$ của mô hình tuyến tính; khối 2 thực hiện (??) và (??) cho đến khi dấu dự đoán trùng nhãn; khối 3 chạy một lần với trọng số ngẫu nhiên; khối 4 và khối 5 chỉ đổi w thành đường thẳng và chiếu lại lịch sử các lần sai.